FILE: C:\HCL_TRAINING\server\src\controllers\screeningController.js

LINES: 213 | SIZE: 6.2 KB | LANGUAGE: JS

MODULE: Screening Results Retrieval, Manual Overrides, and Comparison API

```
1  | const ScreeningResult = require('../models/ScreeningResult');
2  | const Candidate = require('../models/Candidate');
3  | const Job = require('../models/Job');
4  | const Resume = require('../models/Resume');
5  | const ScoringConfig = require('../models/ScoringConfig');
6  | const { calculateCandidateMatch } = require('../services/matchingEngine/scoringEngine');
7  | const { logActivity } = require('../services/auditLogger');
8  |
9  | // @desc    Get screening results for a job
10 | // @route   GET /api/screening/job/:jobId
11 | const getJobScreeningResults = async (req, res, next) => {
12 |   try {
13 |     const { jobId } = req.params;
14 |     const { minScore } = req.query;
15 |
16 |     const filter = {
17 |       jobId,
18 |       organization: req.organizationId,
19 |     };
20 |
21 |     if (minScore) {
22 |       filter.overallScore = { $gte: Number(minScore) };
23 |     }
24 |
25 |     const results = await ScreeningResult.find(filter)
26 |       .sort({ overallScore: -1 })
27 |       .populate('candidateId')
28 |       .populate('resumeId', 'fileName qualityScore qualityIssues isDuplicate');
29 |
30 |     res.status(200).json({
31 |       success: true,
32 |       count: results.length,
33 |       data: results,
34 |     });
35 |   } catch (error) {
36 |     next(error);
37 |   }
38 | };
39 |
40 | // @desc    Get single screening result detail
41 | // @route   GET /api/screening/:id
42 | const getScreeningResultById = async (req, res, next) => {
43 |   try {
44 |     const result = await ScreeningResult.findOne({
45 |       _id: req.params.id,
46 |       organization: req.organizationId,
47 |     })
48 |       .populate('candidateId')
49 |       .populate('jobId')
50 |       .populate('resumeId')
51 |       .populate('notes.author', 'name email avatar');
52 |
53 |     if (!result) {
54 |       return res.status(404).json({ success: false, message: 'Screening result not found
   > ' });
55 |     }
56 |
57 |     res.status(200).json({
58 |       success: true,
59 |       data: result,
60 |     });
61 |   } catch (error) {
62 |     next(error);
63 |   }
64 | };
65 |
```

```
66 |  // @desc    Update recruiter manual override (Module E)
67 |  // @route   PATCH /api/screening/:id/override
68 |  const overrideScreeningScore = async (req, res, next) => {
69 |    try {
70 |      const { adjustedScore, reason } = req.body;
71 |
72 |      if (typeof adjustedScore !== 'number' || adjustedScore < 0 || adjustedScore > 100) {
73 |        return res.status(400).json({ success: false, message: 'Please provide a valid adj
   >  usted score between 0 and 100' });
74 |      }
75 |
76 |      if (!reason || !reason.trim()) {
77 |        return res.status(400).json({ success: false, message: 'Recruiter justification re
   >  ason is required for auditability' });
78 |      }
79 |
80 |      const result = await ScreeningResult.findOne({
81 |        _id: req.params.id,
82 |        organization: req.organizationId,
83 |      });
84 |
85 |      if (!result) {
86 |        return res.status(404).json({ success: false, message: 'Screening result not found
   >  ' });
87 |      }
88 |
89 |      const originalScore = result.recruiterOverride.isOverridden
90 |        ? result.recruiterOverride.originalScore
91 |        : result.overallScore;
92 |
93 |      result.recruiterOverride = {
94 |        isOverridden: true,
95 |        originalScore: originalScore,
96 |        adjustedScore: adjustedScore,
97 |        reason: reason,
98 |        overriddenBy: req.user._id,
99 |        overriddenAt: new Date(),
100 |      };
101 |
102 |      result.overallScore = adjustedScore;
103 |      await result.save();
104 |
105 |      await logActivity({
106 |        organizationId: req.organizationId,
107 |        userId: req.user._id,
108 |        action: 'CANDIDATE_SCORE_OVERRIDDEN',
109 |        entityType: 'ScreeningResult',
110 |        entityId: result._id,
111 |        details: { previousScore: originalScore, newScore: adjustedScore, reason },
112 |        req,
113 |      });
114 |
115 |      res.status(200).json({
116 |        success: true,
117 |        data: result,
118 |      });
119 |    } catch (error) {
120 |      next(error);
121 |    }
122 |  };
123 |
124 |  // @desc    Compare 2 to 4 candidates side-by-side (Module F - Candidate Comparison Stud
   >  io)
125 |  // @route   POST /api/screening/compare
126 |  const compareCandidates = async (req, res, next) => {
127 |    try {
128 |      const { candidateIds, jobId } = req.body;
129 |
130 |      if (!candidateIds || !Array.isArray(candidateIds) || candidateIds.length < 2 || cand
   >  idateIds.length > 4) {
131 |        return res.status(400).json({
132 |          success: false,
```

```
133 |         message: 'Please select between 2 and 4 candidates for comparison',
134 |       });
135 |     }
136 |
137 |     if (!jobId) {
138 |       return res.status(400).json({ success: false, message: 'Job opening context is req
    >   uired for candidate comparison' });
139 |     }
140 |
141 |     const job = await Job.findOne({ _id: jobId, organization: req.organizationId });
142 |     if (!job) {
143 |       return res.status(404).json({ success: false, message: 'Job opening not found' });
144 |     }
145 |
146 |     // Retrieve candidates and their screening results for this job
147 |     const candidates = await Candidate.find({
148 |       _id: { $in: candidateIds },
149 |       organization: req.organizationId,
150 |     });
151 |
152 |     const screeningResults = await ScreeningResult.find({
153 |       candidateId: { $in: candidateIds },
154 |       jobId: job._id,
155 |       organization: req.organizationId,
156 |     }).populate('resumeId', 'qualityScore fileName');
157 |
158 |     const resultsMap = {};
159 |     screeningResults.forEach((sr) => {
160 |       resultsMap[sr.candidateId.toString()] = sr;
161 |     });
162 |
163 |     // Form comparison cards
164 |     const comparisonCards = candidates.map((cand) => {
165 |       const sr = resultsMap[cand._id.toString()];
166 |       return {
167 |         candidate: {
168 |           _id: cand._id,
169 |           name: cand.name,
170 |           email: cand.email,
171 |           location: cand.location,
172 |           totalExperienceYears: cand.totalExperienceYears,
173 |           skills: cand.skills,
174 |           education: cand.educationTimeline,
175 |         },
176 |         screening: sr || {
177 |           overallScore: 0,
178 |           componentScores: { skillAlignment: 0, semanticSimilarity: 0, experienceScore:
    >   0, qualificationScore: 0 },
179 |           matchedRequiredSkills: [],
180 |           missingRequiredSkills: (job.mandatorySkills || []).map((m) => m.name),
181 |           supportingExcerpts: [],
182 |           dataQualityWarnings: ['Candidate not yet screened for this role'],
183 |         },
184 |       };
185 |     });
186 |
187 |     res.status(200).json({
188 |       success: true,
189 |       data: {
190 |         job: {
191 |           _id: job._id,
192 |           title: job.title,
193 |           department: job.department,
194 |           mandatorySkills: job.mandatorySkills,
195 |           preferredSkills: job.preferredSkills,
196 |           minYearsExperience: job.minYearsExperience,
197 |           qualifications: job.qualifications,
198 |         },
199 |         candidates: comparisonCards,
200 |       },
201 |     });
202 |   } catch (error) {
```

```
203 |       next(error);
204 |     }
205 |   };
206 |
207 |   module.exports = {
208 |     getJobScreeningResults,
209 |     getScreeningResultById,
210 |     overrideScreeningScore,
211 |     compareCandidates,
212 |   };
213 |
```